

VAJRA

Visual Audio Joint-modal Rendering Architecture

VAJRA 2.0

Document 5 of 5

Verification and Test Documentation

What is checked automatically, and what is not

WHO THIS IS FOR

Whoever maintains or audits the platform

WHAT IT COVERS

- Ten suites, 503 assertions, and what each covers
- How to run them — no GPU, no weights, seconds
- What the tests explicitly do NOT cover
- The faults found in trial, each now held by a test

THE FULL SET

- | | |
|--|---------------------------|
| 1. System Requirements and Sizing | specifying the machine |
| 2. Deployment and Operations Guide | installing and running it |
| 3. Operator's User Manual | using it |
| 4. Technical Writeup | understanding it |
| ■ 5. Verification and Test Documentation | maintaining it |

Lt Col Raunak Malik · M.Tech, MCTE Mhow · IIT Indore

Supervisors: Dr. Nagendra Kumar (IIT Indore) · Dr. Krishan Berwal (MCTE Mhow)

Offline synthetic-media platform · for authorised research and capability development

1. Running Them

The whole suite runs on CPU, needs no model weights and no GPU, and finishes in seconds. That is what allows it to gate every change rather than being run occasionally.

```
cd /opt/vajra
for f in tests/test_*.py; do python3 "$f" || echo "FAILED: $f"; done
```

Each suite prints one line per assertion and exits non-zero on failure, so the loop above is enough for a build gate. No test framework is required.

Note. Verified before this handover: **10 suites, 503 assertions, all passing** on the 2.0 line.

2. What Each Suite Covers

Suite	Assertions	What it holds
test_dsp.py	83	The signal processing: loudness matching, spectral matching, time-scaling, crossfading, noise-floor estimation. Numerical, with known inputs and expected outputs.
test_vision.py	96	Frame handling, face-region geometry, window selection and compositing arithmetic.
test_router_scorecard.py	74	Tier selection under every combination of language, licence and installed engine; and the scorecard's percentile mathematics.
test_relip_integration.py	56	The editing pipeline end to end with stubbed models: diff, route, fit, splice, report.
test_viitor.py	53	The local-infill engine's readiness checks, request construction and failure messages.
test_textdiff.py	49	Word-level differencing between original and edited transcripts, including the awkward cases: insertions at a boundary, repeated words, punctuation-only changes.
test_app_ui.py	25	That every control the manual documents exists, with the range and default the manual states.
test_fit_regression.py	21	The duration-fitting logic that caused the worst fault found in trial — see section 4.
test_notebook.py	18	That the notebook's cells parse and its steps stay in order.
test_setup_scripts.py	28	The installer: shared steps stay behind the lock, every offered target maps to a real script, and the host path runs preflight before it downloads anything.

3. What These Tests Do NOT Cover

Caution. Read this section before relying on a green run. A suite that appears to pass everything while silently skipping the hard parts actively misleads whoever maintains the system next.

Four things are outside the reach of a CPU-only suite, and all four are in the part of the system that does the actual work:

Not covered	Why	How it is covered instead
Model loading	Needs the weights and a GPU	GPU trial on the host; the first run of each tab is the test
Actual speech synthesis	Needs the voice models resident	Listening to the result, and the detectability scorecard

Not covered	Why	How it is covered instead
Actual lip-sync rendering	Needs the diffusion engine on a GPU	Watching the output; the report states how much was re-rendered
End-to-end perceptual quality	Not reducible to an assertion	The scorecard measures the join; a human judges the content

The scorecard itself carries the same discipline: it names four metrics it does **not** measure — speaker similarity, naturalness, intelligibility and lip-sync accuracy — beside the ones it does.

4. Faults Found in Trial, Now Held by Tests

Every one of these passed the offline suite and still failed on hardware. That is the argument for trial in one line — and each is now a permanent regression test, so it cannot return.

Symptom in trial	Root cause	Now held by
An edit mixed new words with the old recording	The replacement was longer than its slot; the stretch hit its cap, the remainder was trimmed, and the original resumed mid-phrase — 0.48 s of a 0.96 s phrase discarded	test_fit_regression.py
The edited span sounded noisy and over-processed	Measured: not the separation model as first suspected, but the matching chain stacking gain on audio that needed none	test_dsp.py, test_relip_integration.py
An environment build failed on one of three parallel targets	Concurrent installs writing the same system package directory	test_setup_scripts.py
The voice service reported ready, then refused the request	Readiness polled a gateway that answers before the services behind it have loaded	test_viitor.py
A better tier was available but the message implied otherwise	The router reported only the first blocker it found, not all of them	test_router_scorecard.py

Two of these were diagnosed by measuring the output rather than by reading the code, and one overturned the initial hypothesis. That is the habit worth inheriting along with the software.

5. If You Change Something

- **Run the suite before and after.** It takes seconds and it is the project's only gate.
- **If you fix a fault, add an assertion for it.** The suite grew from real failures rather than from imagination, and that is why it is worth running.
- **If a test fails, read it before changing it.** On this project a failing test has more often been right than the code — but not always: one test in this very suite asserted on a header comment rather than on behaviour, and was itself the bug.
- **Do not weaken a test to make it pass.** If the assertion no longer reflects intent, replace it with one that does, and say so in the commit.

6. Reproducing the Measured Results

The figures quoted in the technical writeup and the presentations are reproducible on any machine, with no GPU:

Claim	Figure	Where it comes from
Loudness match	9.60 dB → 0.02 dB	The same edit measured with the correction chain off, then on

Claim	Figure	Where it comes from
Spectral (tone) match	29.5 dB → 6.8 dB	As above
Seam discontinuity	35.4 dB → 10.1 dB	As above
Audio outside an edit	0.00e+00 difference	Bit-exact pass-through test
Proportion of a line regenerated	100% → 35%	Word-level span resolution
Frames re-rendered on a 10-minute clip	0.483%	Windowed lip-sync

The controlled experiments use pinned random seeds. The scorecard is self-referential by design, so its baseline changes with the material — that is intended behaviour, not variance.